Universidad Simón Bolívar
Departamento de Computación y Tecnología de la Información
CI-3661 — Laboratorio de Lenguajes de Programación I
Septiembre – Diciembre 2026

Prof. Leonardo López Almazán
leonardolopez@usb.ve

Proyecto 1
Motor de aventuras de texto en Haskell

35 puntos

Fechas

Publicación:	viernes 6 de noviembre de 2026 (semana 6), después de la clase 6
Entrega:	**jueves 19 de noviembre de 2026, 23:59** (semana 8)
Entrega tardía:	hasta el domingo 22 de noviembre de 2026, 23:59, con descuento
Modalidad:	**en pareja** (los dos integrantes en la misma sección)

1. Resumen

Van a construir un **motor** para juegos de aventura de texto: un programa que lee la descripción de un mundo (salas, objetos, salidas y una meta) desde un archivo, comprueba que el mundo tenga sentido, y deja que un jugador lo recorra escribiendo comandos en español. El motor no conoce ninguna historia: debe poder ejecutar *cualquier* archivo que respete el formato de la sección 3.

Lo que se evalúa es el diseño funcional: modelar el dominio con tipos algebraicos, mantener la lógica del juego **pura** y separada de la entrada y salida, construir un reconocedor con combinadores propios y reportar errores con `Either`. También se evalúan sus **decisiones de diseño** y que sepan defenderlas: el proyecto deja muchas cosas abiertas a propósito.

¿Qué clase cubre cada parte?

Todo lo que hace falta ya se vio:

Tipos del dominio, registros, módulos	clase 4 (23 de octubre)
`Foldable`, `Monoid`, reconocedores con `ReadS`	clase 5 (30 de octubre)
E/S, `Functor`/`Applicative`/`Monad`, `readMaybe`	clase 6 (6 de noviembre)

La clase 7 (evaluación perezosa, 13 de noviembre) solo hace falta para una de las decisiones opcionales de `DECISIONES.md`.

2. El juego

Al arrancar, el motor muestra el título del mundo y la descripción de la sala inicial. Luego repite: muestra un indicador (`> `), lee una línea, la interpreta como comando y muestra el resultado. El jugador gana cuando se cumple la meta del mundo.

2.1 Comandos mínimos

Comando	Efecto
`ir <dirección>`	Mueve al jugador por esa salida, si existe y no está bloqueada. Las direcciones son `norte`, `sur`, `este`, `oeste`, `arriba` y `abajo`.
`mirar`	Describe la sala actual: su descripción, los objetos que hay y las salidas.
`tomar <objeto>`	Pasa un objeto de la sala al inventario. Sinónimo: `coger`.
`soltar <objeto>`	Deja un objeto del inventario en la sala actual.
`examinar <objeto>`	Muestra la descripción de un objeto del inventario o de la sala.
`inventario`	Lista lo que lleva el jugador. Sinónimo: `inv`.
`guardar <archivo>`	Guarda la partida en un archivo.
`cargar <archivo>`	Recupera una partida guardada.
`ayuda`	Muestra los comandos disponibles.
`salir`	Termina el programa.

Los objetos se nombran por su identificador (`tomar memoria_usb` siempre debe funcionar). Si aceptan otras formas (`tomar la memoria usb`, `n` por `norte`, mayúsculas…) es una decisión suya y va en `DECISIONES.md`.

Una **salida bloqueada** es la que declara `requiere <objeto>`: solo se puede cruzar si el jugador lleva ese objeto en el inventario. Si no lo lleva, el motor lo dice y el jugador no se mueve.

3. Formato del archivo de mundo

- El archivo está en UTF-8. Las líneas vacías y las que empiezan con `#` (comentarios) se ignoran. Se ignoran también los espacios al inicio y al final de cada línea.
- El archivo es una secuencia de **bloques** separados por líneas que solo contienen `--`. Cada línea de un bloque tiene la forma `CLAVE: valor`, con la clave en mayúsculas.
- El **primer bloque** es la cabecera. Los demás son bloques `OBJETO` o `SALA`, en cualquier orden, según su primera línea.
- Los **identificadores** de salas y objetos son palabras formadas por letras minúsculas (incluidas `á é í ó ú ñ ü`), dígitos y `_`. No tienen espacios. Las salas y los objetos tienen espacios de nombres separados.

Bloque y clave	Valor	¿Cuántas?
Cabecera		
`MUNDO`	título (texto libre)	exactamente una
`INICIO`	identificador de sala	exactamente una
`META`	`llegar a <sala>` o `llevar <objeto> a <sala>`	exactamente una
Bloque de objeto		
`OBJETO`	identificador (primera línea)	exactamente una
`DESC`	descripción (texto libre)	exactamente una
Bloque de sala		
`SALA`	identificador (primera línea)	exactamente una
`DESC`	descripción (texto libre)	exactamente una
`SALIDA`	`<dirección> -> <sala>`, o bien `<dirección> -> <sala> requiere <objeto>`	cero o más
`OBJETO`	identificador de un objeto definido	cero o más

La meta `llegar a <sala>` se cumple al entrar en esa sala; `llevar <objeto> a <sala>` se cumple al estar en la sala con el objeto en el inventario. Las salidas *no* son bidireccionales: si se puede volver, el archivo lo dice. Las variantes (sección 7) agregan algunas claves más.

Ejemplo (fragmento del archivo `mundos/entrega.txt` del material):

```
MUNDO: Entrega a última hora
INICIO: estacionamiento
META: llevar proyecto_impreso a oficina_del_profesor
---
OBJETO: carnet
DESC: Tu carnet estudiantil. La foto es de primer año y no te pareces.
---
SALA: plaza
DESC: La plaza del rectorado. Un gato duerme sobre un banco.
SALIDA: sur -> estacionamiento
SALIDA: norte -> pasillo_mys
SALIDA: oeste -> biblioteca requiere carnet
```

4. Validación

Un archivo puede estar bien escrito y aun así describir un mundo imposible. El motor debe detectar **todos** los errores de esta lista y reportarlos **todos juntos**, no solo el primero:

V1. Errores de sintaxis: clave desconocida, línea sin la forma `CLAVE: valor`, campo obligatorio ausente o repetido, `SALIDA` o `META` mal formadas, dirección inexistente, identificador inválido. Indiquen el número de línea.

V2. Dos salas o dos objetos con el mismo identificador.

V3. Una salida hacia una sala que no existe.

V4. Una salida que requiere un objeto que no existe.

V5. Una sala con dos salidas en la misma dirección.

V6. Una sala que contiene un objeto no definido.

V7. Un objeto colocado en más de una sala (o dos veces en la misma).

V8. `INICIO` o `META` con una sala o un objeto inexistentes.

V9. Una sala que no es alcanzable desde `INICIO` siguiendo salidas (para esto se ignoran los requisitos).

V10. Un objeto que se necesita (para la meta o para cruzar una salida) pero no está colocado en ninguna sala.

Es razonable que, si hay errores de sintaxis (V1), no se intente validar lo demás. Los textos de los mensajes son libres, pero cada uno va en su línea, empieza con `Error:` y nombra lo que está mal. El material trae `mundos/invalido_semantica.txt`, que tiene nueve errores de V2 a V10, y `mundos/invalido_sintaxis.txt`.

5. Arquitectura

El proyecto es un paquete de `cabal` o de `stack` con, como mínimo, estos módulos. Los nombres de los módulos y de las funciones de esta tabla son **obligatorios**; la forma de los tipos es decisión suya.

Módulo	Contenido
`Motor.Tipos`	Los tipos del dominio: `Mundo`, `Sala`, `Objeto`, `Direccion`, `Meta`, `Estado`, `Comando`, `ErrorMundo`…
`Motor.Reconocedor`	Sus combinadores de reconocimiento, construidos por ustedes sobre `ReadS` (directamente o envuelto en un `newtype` con sus instancias).
`Motor.Mundo`	`leerMundo :: String -> Either [ErrorMundo] Mundo` Recibe el *texto* del archivo; reconoce y valida. Es pura.
`Motor.Comandos`	`leerComando :: String -> Maybe Comando` Pura.

`Motor.Nucleo`	`estadoInicial :: Mundo -> Estado`
	`ejecutar :: Comando -> Estado -> ([String], Estado)`
	La segunda da los mensajes para el jugador y el estado nuevo. Pura.

`Motor.Persistencia`	Todo lo que toca archivos: leer el mundo, guardar y cargar partidas.
`app/Main.hs`	Argumentos de la línea de comandos y el bucle de juego.

Restricciones

- Ni `IO` ni `unsafePerformIO` fuera de `Motor.Persistencia` y `Main`. Un `IO` en el núcleo es un error grave.
- El mundo se reconoce con **sus** combinadores. No se permiten `parsec`, `megaparsec`, `attoparsec`, `ReadP`, `Text.Read.Lex` ni expresiones regulares. Sí pueden usar `lines`, `words` y las funciones de `Data.Char` y `Data.List`.
- Paquetes permitidos: `base`, `containers`, `directory`, `filepath` y, solo para las pruebas, `HUnit`, `hspec` o `QuickCheck`. Cualquier otro, pregunten antes.
- Funciones parciales (`head`, `fromJust`, `(!!)`, `read`) solo si pueden justificar en un comentario por qué nunca fallan ahí.

6. Interfaz del programa

El ejecutable se llama `aventura`. Estas cosas son fijas porque las pruebas automáticas del profesor dependen de ellas:

- `aventura MUNDO` carga el mundo y empieza el juego. Si el mundo tiene errores, los imprime (uno por línea, empezando con `Error:`) y termina con código de salida 1.
- `aventura -validar MUNDO` solo valida: imprime `Mundo válido.` y termina con código 0, o imprime los errores y termina con código 1.
- Al cumplirse la meta, el motor imprime la línea `*** ¡Has ganado! ***` y termina con código 0.
- El programa debe funcionar con la entrada redirigida desde un archivo: `aventura mundo.txt < guion.txt`. Al llegar al fin de la entrada, termina normalmente (sin excepciones).
- Si el archivo de partida que se quiere cargar no existe, está corrupto o es de otro mundo, el programa lo informa y sigue; nunca se cae.

Todo lo demás (los mensajes, el formato de `mirar`, el indicador) es libre. Con el mundo de ejemplo y el guion `guiones/entrega.txt` del material, el programa debe terminar ganando.

7. Variantes

Cada pareja implementa **una** variante. Para saber cuál, sumen sus dos carnets como números (sin el guion) y tomen el resto de dividir entre 4. Por ejemplo, `19-10437` y `20-10555` dan $1910437 + 2010555 = 3920992$, y $3920992 \bmod 4 = 0$. Quien trabaje solo usa su propio carnet.

0 — Oscuridad.
Una sala puede tener `OSCURA: si` y un objeto `LUZ: si`. En una sala oscura, si el jugador no lleva un objeto que dé luz, `mirar` no muestra objetos ni salidas y no se puede tomar ni examinar nada de la sala. Ustedes deciden si se puede salir de una sala oscura a ciegas, y por dónde.

1 — Peso.
La cabecera tiene `CAPACIDAD: n` y cada objeto puede tener `PESO: n` (enteros positivos; si no se da, el peso es 1). El jugador no puede llevar más peso que la capacidad. Validación adicional: ningún objeto pesa más que la capacidad, y los objetos de la meta y de los requisitos deben poder llevarse a la vez cuando haga falta (decidan qué tan lejos llevan esta comprobación y justifíquenlo).

2 — Turnos.
La cabecera tiene `TURNOS: n`. Ciertos comandos gastan un turno; si se acaban antes de cumplir la meta, el motor imprime `*** Se acabó el tiempo ***` y termina con código 0. Ustedes deciden qué comandos gastan turnos (¿`mirar`? ¿un comando que falla?) y cómo se muestran los turnos restantes. Al cargar una partida se recuperan los turnos que tenía.

3 — Combinar.
La cabecera puede tener líneas `RECETA: a + b -> c`. El comando `combinar a con b` reemplaza en el inventario los objetos `a` y `b` por `c`. Validación adicional: los tres objetos existen, y V10 cuenta como conseguible un objeto que se obtiene por receta. Ustedes deciden si el orden de `a` y `b` importa y si `c` puede estar además en alguna sala.

El motor puede rechazar como desconocidas las claves de las variantes que no les tocaron.

8. Lo que deben entregar

8.1 Su propio mundo

- `mundos/mundo.txt`: un mundo original con al menos 8 salas, 6 objetos y 2 salidas bloqueadas, que use su variante de forma que importe para ganar.
- `guiones/mundo.txt`: un guion con el que se gana ese mundo.
- `mundos/invalidos/`: al menos 3 mundos inválidos que, entre todos, provoquen al menos 6 errores distintos de la sección 4, incluido uno propio de su variante. Cada archivo empieza con un comentario que lista los errores que debe dar.

8.2 Pruebas

Una `test-suite` en el archivo `.cabal` que corra con `cabal test` o `stack test`, con al menos 15 casos, entre ellos:

- `leerMundo` sobre el mundo de ejemplo, sobre el suyo y sobre cada uno de sus mundos inválidos, comprobando la cantidad de errores;
- secuencias de comandos sobre `ejecutar` (sin `IO`) que lleguen a la meta, choquen con una salida bloqueada y ejerciten su variante;
- casos de `leerComando`, incluidos comandos inválidos.

8.3 Decisiones de diseño: `DECISIONES.md`

Este documento vale tanto como una parte del código. Debe tener **al menos seis decisiones** en formato ADR (*Architecture Decision Record*), cada una con:

```
## D3. Cómo se representan las salidas bloqueadas
Contexto:      qué problema había que resolver y por qué no era obvio.
Opciones:      al menos dos alternativas reales que consideraron.
Decisión:      cuál eligieron.
Consecuencias: qué ganaron, qué perdieron, qué harían distinto.
```

Las decisiones marcadas con ⋆ son obligatorias; el resto, escojan:

- ⋆ ¿Cómo representan el mundo y los identificadores? (`Map` o listas, `String` o `newtype`, ¿qué invariantes garantiza el tipo y cuáles no?)
- ⋆ ¿Qué guarda el `Estado`? ¿Se copia el mundo entero o solo lo que cambia? ¿Dónde quedan los objetos soltados?
- ⋆ ¿Cómo logran reportar *todos* los errores de validación y no solo el primero? ¿Por qué no basta con encadenar `Either` con `do`?
- ⋆ ¿Cómo diseñaron el reconocedor? (`ReadS` directo o `newtype`; ¿qué hacen con la ambigüedad, es decir, con varios resultados?; ¿cómo obtienen el número de línea de un error?)
- ⋆ Las decisiones propias de su variante.

1. Qué formas de los comandos aceptan y por qué (sinónimos, artículos, objetos con espacios, abreviaturas).
2. El formato de las partidas guardadas: `Show`/`Read` del estado completo o solo de los cambios; qué pasa si el mundo cambió entre guardar y cargar.

3. Cómo se comporta el programa con la entrada redirigida y con el fin de la entrada (`getLine` frente a `getContents`, `isEOF`...), y qué papel juega ahí la evaluación perezosa (clase 7).
4. Si una salida bloqueada queda abierta para siempre después de cruzarla.
5. Cualquier otra decisión que les haya costado tomar.

Una decisión sin alternativas reales ("usamos `Map` porque es mejor") no cuenta, y una que no coincide con el código tampoco. Cada decisión indica quién la redactó, y deben estar repartidas entre los dos integrantes. Escriban en sus palabras y refiéranse a su propio código (módulos, funciones, ejemplos concretos).

8.4 Declaración de uso de IA: `IA.md`

Pueden usar asistentes de IA. En `IA.md` declaren qué herramientas usaron, para qué partes y con qué propósito, y den **al menos un ejemplo concreto** de una sugerencia que rechazaron o tuvieron que corregir, y por qué. Si no usaron ninguna, díganlo. Declarar el uso no resta; presentar como propio código que no escribieron, sí.

8.5 Repositorio

- Un repositorio **público** en GitHub. Envíen el enlace a `leonardolopez@usb.ve` apenas lo creen, con los nombres y carnets de la pareja. Como es público, copiar de otro repositorio es plagio, y el historial de commits muestra quién escribió cada cosa primero.
- El historial debe mostrar trabajo incremental de **los dos** integrantes: varios commits con mensajes que digan qué cambió, que permitan ver cómo se construyó el proyecto. No importa cuándo trabajen, pero un único commit con todo el proyecto se penaliza. La nota del proyecto es la misma para los dos, salvo que el historial muestre que uno de ellos no participó; en ese caso, su nota se reduce en proporción.
- `README.md` con los nombres, carnets, variante (con la cuenta), cómo compilar, correr y probar.
- Una etiqueta (`git tag`) `entrega` marca la versión que se califica, la que apunte a ella a la hora límite.

9. Evaluación

Aspecto	Puntos
Modelado con tipos (`Motor.Tipos`)	4
Reconocedor del mundo con combinadores propios	5
Validación completa (V1–V10, todos los errores)	5
Núcleo puro y comandos	6
Variante	3.5
E/S: interfaz, persistencia, entrada redirigida	3.5
Pruebas y mundos propios	3
`DECISIONES.md`, `IA.md` e historial	5
Total	35

Además de sus pruebas, el motor se corre con pruebas automáticas y con mundos que ustedes no

conocen, así que vale la pena probar casos raros del formato.

Entregas tardías

Se aceptan entregas hasta el **domingo 22 de noviembre, 23:59**, con un descuento de 5 puntos por cada día de retraso o fracción. Después no se recibe el proyecto. El retraso se cuenta desde la fecha del commit al que apunta la etiqueta `entrega`.

10. Material

- `mundos/entrega.txt`: el mundo de ejemplo.
- `guiones/entrega.txt`: un guion que gana el mundo de ejemplo.
- `mundos/invalido_semantica.txt` y `mundos/invalido_sintaxis.txt`.